PRACTICAL-4

Aim: To develop a game-playing agent for Tic-Tac-Toe using the Minimax algorithm and Alpha-Beta Pruning.

Theory:

Minimax Algorithm: Minimax is a decision-making algorithm used in two-player games. It selects the best move by maximizing the computer's score and minimizing the opponent's score.

Alpha-Beta Pruning: Alpha-Beta Pruning is an optimization technique that eliminates unnecessary branches in the Minimax search tree, reducing the number of moves evaluated.

In this program, the human plays as X, and the computer plays as O. The computer evaluates possible moves and selects the best move to win or avoid losing.

Program/Code:

```
import math

# Print the board
def print_board(board):
    print("\n")
    print(" " + board[0] + " | " + board[1] + " | " + board[2])
    print("---+---+---")
    print(" " + board[3] + " | " + board[4] + " | " + board[5])
    print("---+---+---")
    print(" " + board[6] + " | " + board[7] + " | " + board[8])
    print("\n")


# Check if there is a winner
def check_winner(board):
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]

    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] and board[a] != " ":
            return board[a]
```

```
    if " " not in board:
        return "Draw"

    return None


# Minimax with Alpha-Beta Pruning
def minimax(board, depth, is_maximizing, alpha, beta):
    result = check_winner(board)

    if result == "O":
        return 10 - depth

    if result == "X":
        return depth - 10

    if result == "Draw":
        return 0

    # Maximizing player: Computer (O)
    if is_maximizing:
        best_score = -math.inf

        for i in range(9):
            if board[i] == " ":
                board[i] = "O"

                score = minimax(
                    board, depth + 1, False, alpha, beta
                )

                board[i] = " "

                best_score = max(best_score, score)
                alpha = max(alpha, best_score)

                if beta <= alpha:
                    break

        return best_score

    # Minimizing player: Human (X)
    else:
        best_score = math.inf
```

```
        for i in range(9):
            if board[i] == " ":
                board[i] = "X"

                score = minimax(
                    board, depth + 1, True, alpha, beta
                )

                board[i] = " "

                best_score = min(best_score, score)
                beta = min(beta, best_score)

                if beta <= alpha:
                    break

        return best_score


# Find the best move for the computer
def best_move(board):
    best_score = -math.inf
    move = -1

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"

            score = minimax(
                board, 0, False, -math.inf, math.inf
            )

            board[i] = " "

            if score > best_score:
                best_score = score
                move = i

    return move


# Main game
def play_game():
    board = [" "] * 9

    print("====================================")
```

```
print("     TIC-TAC-TOE AI GAME")
print("     Minimax + Alpha-Beta Pruning")
print("=====================================")

print("\nYou are X")
print("Computer is O")

print("\nBoard positions:")
print(" 1 | 2 | 3")
print("---+---+---")
print(" 4 | 5 | 6")
print("---+---+---")
print(" 7 | 8 | 9")

while True:
    # Human's turn
    print_board(board)

    while True:
        try:
            position = int(input("Enter your move (1-9): "))

            if position < 1 or position > 9:
                print("Please enter a number from 1 to 9.")
                continue

            if board[position - 1] != " ":
                print("That position is already occupied.")
                continue

            break

        except ValueError:
            print("Please enter a valid number.")

    board[position - 1] = "X"
    result = check_winner(board)

    if result is not None:
        print_board(board)

        if result == "X":
            print("You win!")
        elif result == "Draw":
            print("It's a draw!")
        break
```

```
    # Computer's turn
    print("Computer is thinking...")

    move = best_move(board)
    board[move] = "O"

    print("Computer chose position:", move + 1)

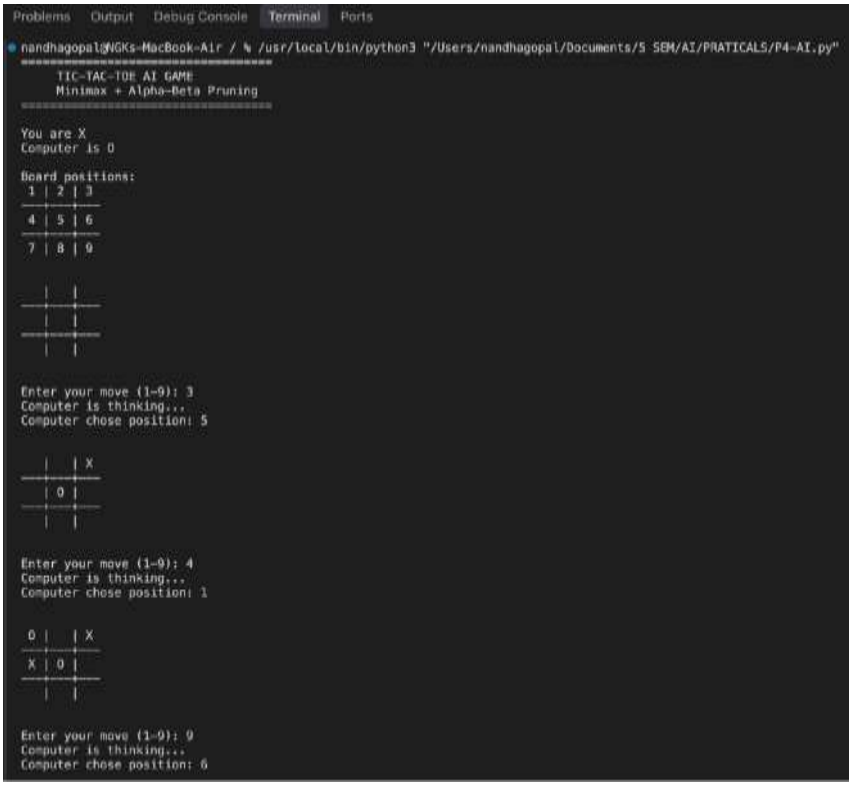
    result = check_winner(board)

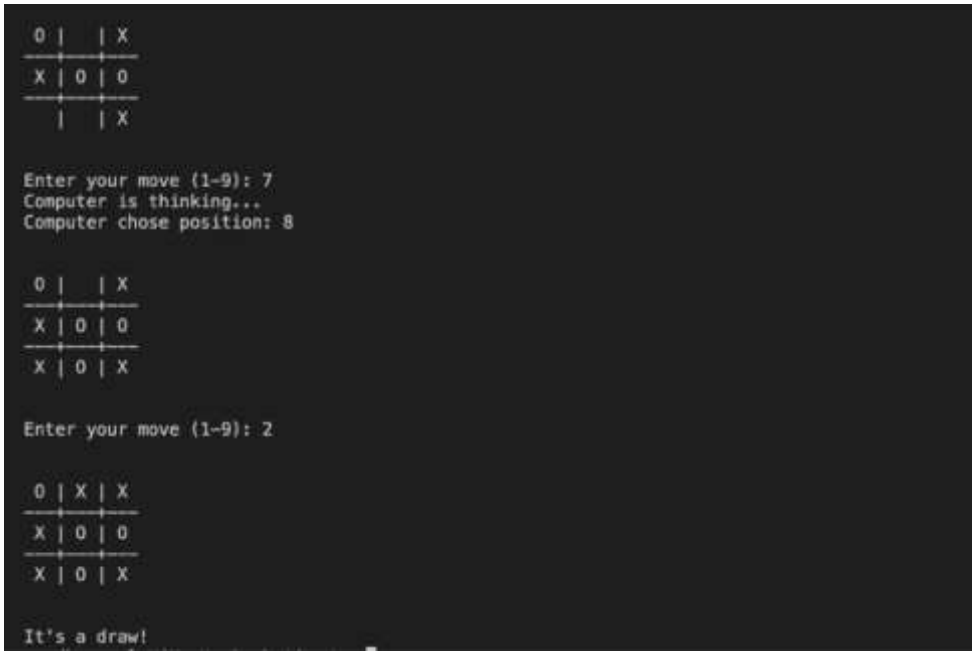
    if result is not None:
        print_board(board)

        if result == "O":
            print("Computer wins!")
        elif result == "Draw":
            print("It's a draw!")
        break


# Run the game
if __name__ == "__main__":
    play_game()
```

Output:





Conclusion:

Thus, a Tic-Tac-Toe game-playing agent was developed successfully using the Minimax algorithm and Alpha-Beta Pruning. The computer evaluates possible moves and selects the best available move while reducing unnecessary searches.